



Instituto Infnet

2º Ciclo - Fundamentos do Processamento de Dados

Python para Processamento de Dados [26E2_4]

Listas (Lists) - Continuação

Matrizes

Podemos criar matrizes incluindo listas dentro de listas

$$\begin{matrix}
 & \begin{matrix} 1 & 2 & \dots & n \end{matrix} \\
 \begin{matrix} 1 \\ 2 \\ 3 \\ \vdots \\ m \end{matrix} & \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ a_{31} & a_{32} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}
 \end{matrix}$$

```
# - -
# | 3  2 |
# | 6  1 |
# - -
```

```
matriz = [[3, 2], [6, 1]]

for indice_linha in range(len(matriz)):
    for indice_coluna in range(len(matriz)):
        print(matriz[indice_linha][indice_coluna], end=' ')

    print('')
```

```
3 2
6 1
```

```
help(print)
```

Help on built-in function print in module builtins:

```
print(*args, sep=' ', end='\n', file=None, flush=False)
    Prints the values to a stream, or to sys.stdout by default.

    sep
        string inserted between values, default a space.
    end
        string appended after the last value, default a newline.
    file
        a file-like object (stream); defaults to the current sys.stdout.
    flush
        whether to forcibly flush the stream.
```

Exercício 1): Crie um algoritmo que calcula o produto de matrizes (obs: verifique se é possível multiplicar)
Exercício 2): Crie um algoritmo que calcula a soma dos elementos da diagonal principal (obs: elementos com o mesmo índice)

Tuplas (Tuples)

Tuplas são coleções ordenadas e imutáveis, permitindo itens duplicados. Uma vez criada, uma tupla não pode ser modificada.

```
# Criação de Tuplas
coordenadas = (10, 20)
print(coordenadas, type(coordenadas))
print(coordenadas, len(coordenadas))
```

```
(10, 20) <class 'tuple'>
(10, 20) 2
```

```
cores = ('vermelho', 'azul', 'branco')
print(cores, type(cores))
print(cores, len(cores))
```

```
('vermelho', 'azul', 'branco') <class 'tuple'>
('vermelho', 'azul', 'branco') 3
```

```
misturada = (12, 'Python', True)
print(misturada, type(misturada))
print(misturada, len(misturada))
```

```
(12, 'Python', True) <class 'tuple'>
(12, 'Python', True) 3
```

```
# Acessando Elementos, usando o operador []
misturada = (12, 'Python', True)

print(f'Primeiro elemento da tupla: {misturada} - {misturada[0]}')
print(f'Último elemento da tupla: {misturada} - {misturada[-1]}')
```

```
Primeiro elemento da tupla: (12, 'Python', True) - 12
Último elemento da tupla: (12, 'Python', True) - True
```

```
# Observacao
```

```
singleton = (22,)  
print(singleton, type(singleton))  
print(singleton, len(singleton))  
  
singleton[0] = 12
```

```
(22,) <class 'tuple'>  
(22,) 1
```

 Execution error

TypeError: 'tuple' object does not support item assignment

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[37], line 7  
      4 print(singleton, type(singleton))  
      5 print(singleton, len(singleton))  
----> 7 singleton[0] = 12  
  
TypeError: 'tuple' object does not support item assignment
```

Hide error details

```
# Observacao
```

```
# Criando tuplas com a funcao tuple()
```

```
tupla_de_listas = tuple([1, 2, 3])  
print(tupla_de_listas, type(tupla_de_listas))  
print(tupla_de_listas, len(tupla_de_listas))
```

```
# criando tuplas sem parenteses (empacotamento)
```

```
peessoa = 'Gesiel Lopes', 46, 'Professor'  
print(peessoa, type(peessoa))  
print(peessoa, len(peessoa))
```

```
(1, 2, 3) <class 'tuple'>  
(1, 2, 3) 3  
( 'Gesiel Lopes', 46) <class 'tuple'>  
( 'Gesiel Lopes', 46) 2
```

```
# Desempacotamento de Tuplas
```

```
# atribuir elementos a variaveis
```

```
peessoa = 'Gesiel Lopes', 46, 'Professor'  
print(peessoa, type(peessoa))  
print(peessoa, len(peessoa))
```

```
# atribuir os valores da tupla a variaveis
```

```
# forma 1:
```

```
nome = peessoa[0]  
idade = peessoa[1]  
profissao = peessoa[2]
```

```
print(nome, idade, profissao)
```

```
# forma 2:
```

```
nome_, idade_, profissao_ = peessoa  
print(nome_, idade_, profissao_)
```

```
('Gesiel Lopes', 46, 'Professor') <class 'tuple'>  
( 'Gesiel Lopes', 46, 'Professor') 3  
Gesiel Lopes 46 Professor  
Gesiel Lopes 46 Professor
```

```
# Operações com Tuplas
```

```
cores_1 = ('verde', 'amarelo')  
cores_2 = ('azul', 'branco')
```

```
# concatenacao  
todas_cores = cores_1 + cores_2
```

```
print(cores_1, cores_2)  
print(todas_cores)
```

```
# repeticao  
cores_repetidas = cores_1 * 3  
print(cores_1)  
print(cores_repetidas)
```

```
('verde', 'amarelo') ('azul', 'branco')  
('verde', 'amarelo', 'azul', 'branco')  
('verde', 'amarelo')  
('verde', 'amarelo', 'verde', 'amarelo', 'verde', 'amarelo')
```

```
# verificar se um elemento estah presente em uma tupla: operador in  
tem_verde = 'verde' in todas_cores
```

```
# forma 1
```

```
if tem_verde:  
    resultado_da_verificacao_da_cor_verde = 'sim'  
else:  
    resultado_da_verificacao_da_cor_verde = 'nao'
```

```
print(f'Forma 1: Tem verde {resultado_da_verificacao_da_cor_verde}')
```

```
# Forma 2
```

```
# disclaimer: operador ternario do Python: atribuicao condicional: 'sim' se tiver e 'nao' caso contrario  
resultado_da_verificacao_da_cor_verde_ = 'sim' if tem_verde else 'nao'
```

```
print(f'Forma 2: Tem verde {resultado_da_verificacao_da_cor_verde_}')
```

```
Forma 1: Tem verde sim
```

```
Forma 2: Tem verde sim
```

```
# Contagem e localizacao (mesmos metodos da lista)  
contagem = cores_repetidas.count('amarelo')
```

```
print(f'Qtd cor amarela em {cores_repetidas}: {contagem}')
```

```
indice_cor = todas_cores.index('branco')  
print(f'Indice da cor branca em {todas_cores}: {indice_cor}')
```

```
Qtd cor amarela em ('verde', 'amarelo', 'verde', 'amarelo', 'verde', 'amarelo'): 3
```

```
Indice da cor branca em ('verde', 'amarelo', 'azul', 'branco'): 3
```

Aprofundamento

- [Documentação sobre listas](#)
- [Documentação sobre tuplas](#)